Inside the Black Box

Lecture Notes on Mechanistic Interpretability,
In-Context Learning, and the Reversal Curse

Executive Education in AI & Business

Spring 2026

Contents

1 Introduction: Why Look Inside? 2

2 Mechanistic Interpretability: The Birth of AI Neuroscience 2
- 2.1 The Problem: What Do Neurons Represent? . . . 2
- 2.2 Superposition: How Models Pack More Features Than Neurons . . . 3
- 2.3 Sparse Autoencoders: Decomposing the Superposition . . . 4
- 2.4 Circuit Tracing: Following the Reasoning . . . 5
  - 2.4.1 Multi-Hop Reasoning: Dallas → Texas → Austin . . . 5
  - 2.4.2 The Hallucination Circuit . . . 6
  - 2.4.3 The Sycophancy Vector . . . 6
  - 2.4.4 Multilingual Circuits and Abstract Representations . . . 6
- 2.5 Summary: The State of Mechanistic Interpretability . . . 6

3 In-Context Learning and Mesa-Optimizers 7
- 3.1 The Phenomenon: Learning Without Weight Updates . . . 7
- 3.2 The Forward Pass Is Gradient Descent . . . 7
- 3.3 Induction Heads: The Circuit That Enables ICL . . . 9
  - 3.3.1 The Phase Transition . . . 9
- 3.4 Mesa-Optimizers: When the Model Develops Its Own Objectives . . . 10
  - 3.4.1 Objective Misalignment (Pseudo-Alignment) . . . 10
  - 3.4.2 Deceptive Alignment . . . 10
  - 3.4.3 Empirical Evidence . . . 10

4 The Reversal Curse 11
- 4.1 The Phenomenon . . . 11
- 4.2 The Mechanism: Gradient Asymmetry . . . 12
- 4.3 Scaling Does Not Help . . . 13
- 4.4 Mitigation Attempts . . . 13
  - 4.4.1 Data Augmentation . . . 13
  - 4.4.2 Architectural Approaches . . . 14
  - 4.4.3 Retrieval-Augmented Generation (RAG) . . . 14
- 4.5 Implications . . . 14

5 Synthesis: The View From Inside 15

1 Introduction: Why Look Inside?

Large language models (LLMs) have demonstrated remarkable capabilities: passing professional exams, writing coherent essays, generating working code, and holding multi-turn conversations that are often indistinguishable from human interaction. Yet for most of the history of deep learning, we have had essentially *no idea* how these systems actually work internally. We could measure their inputs and outputs, benchmark their performance, and probe their failure modes — but the internal computation remained opaque. Neural networks were, in the most literal sense, black boxes.

This situation has changed dramatically since 2022. A convergence of theoretical insights, engineering breakthroughs, and large-scale empirical studies has opened a window into the internals of transformer-based language models. What we have found is simultaneously more elegant and more alien than anyone expected.

These lecture notes cover three landmark research directions that, taken together, paint a nuanced picture of what is happening inside an LLM:

1. **Mechanistic Interpretability** (§2): We can now identify individual *features* inside a model — directions in activation space that correspond to human-understandable concepts — and trace how they compose into circuits that implement multi-step reasoning. This is the birth of "AI neuroscience."
2. **In-Context Learning and Mesa-Optimizers** (§3): When an LLM is given a few examples in the prompt, it does not merely retrieve stored patterns. It runs an internal learning algorithm — mathematically equivalent to gradient descent — *during inference*, without changing any weights. The model has learned to learn. This raises profound questions about control and alignment.
3. **The Reversal Curse** (§4): Despite the marvels of the previous two sections, LLMs store knowledge in a fundamentally asymmetric way. A model trained on "A is B" will *not* learn "B is A." This is not a scaling problem — it persists at all model sizes. It reveals that the internal knowledge representation of LLMs is profoundly different from human memory.

The unifying thread is simple: *we can now look inside the black box, and what we find is both miraculous and fundamentally alien.*

2 Mechanistic Interpretability: The Birth of AI Neuroscience

2.1 The Problem: What Do Neurons Represent?

The most natural hypothesis about neural network internals is that individual neurons correspond to individual concepts. This idea, sometimes called the *neuron doctrine* by analogy with neuroscience, would make interpretation straightforward: find the "cat neuron," the "legal language neuron," the "Golden Gate Bridge neuron," and so on.

Early work on convolutional neural networks provided some support for this view. Zeiler and Fergus (2014) showed that individual neurons in image classifiers could be visualized and often corresponded to recognizable visual features (edges, textures, object parts). However, as models grew larger and were applied to language, the neuron doctrine began to break down.

The fundamental problem is **polysemanticity**: most individual neurons in a large language model respond to *multiple, seemingly unrelated* concepts. A single neuron might activate for academic citations, Python code, and Korean text. This makes individual neurons essentially uninterpretable.

Definition 2.1 (Polysemanticity)**.** A neuron is *polysemantic* if it activates in response to multiple, semantically distinct input patterns. In contrast, a *monosemantic* neuron activates for a single, coherent concept.

Why does polysemanticity occur? The answer, as discovered by Elhage et al. (2022), lies in a phenomenon called *superposition*.

2.2 Superposition: How Models Pack More Features Than Neurons

Key Insight

The model needs to represent far more concepts than it has neurons. It solves this by encoding each concept as a *direction* in activation space rather than as a single neuron, and packing these directions as near-orthogonal vectors in a space of much lower dimension than the number of concepts.

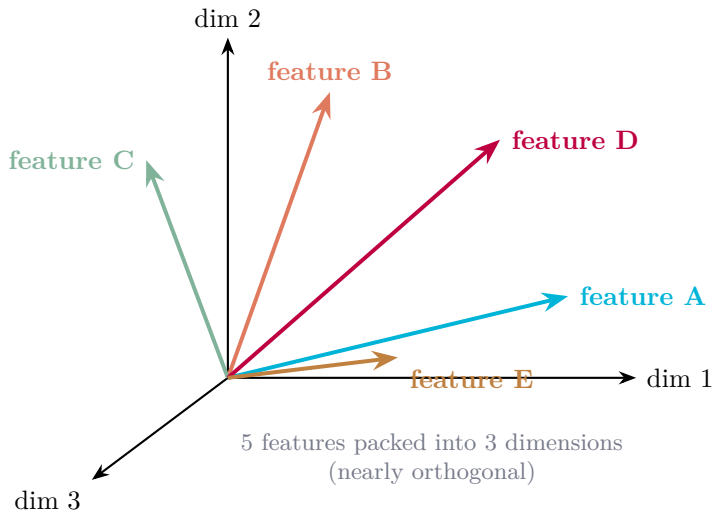
To understand superposition formally, consider a model with d neurons in a given layer. If each concept were assigned to a dedicated neuron, the model could represent at most d concepts. But in practice, the model needs to represent $m \gg d$ features.

The key mathematical insight is that in high-dimensional spaces, it is possible to pack exponentially many *nearly orthogonal* vectors. Two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ are ϵ-nearly orthogonal if $|\langle \mathbf{u}, \mathbf{v} \rangle| < \epsilon$. The Johnson–Lindenstrauss lemma guarantees that $\exp(\Omega(d\epsilon^2))$ such vectors exist.

Proposition 2.1 (Superposition capacity)**.** *In $\mathbb{R}^d$, one can pack $m = \exp(\Omega(d))$ unit vectors such that all pairwise inner products are bounded by $O(\sqrt{(\log m)/d})$. This means a layer with d neurons can, in principle, encode exponentially many features with only small interference between them.*

Elhage et al. (2022) demonstrated this phenomenon using carefully designed "toy models" — small networks trained on synthetic data where the true features were known. They showed that:

1. When the model has more features than dimensions, it *spontaneously* learns to encode features as near-orthogonal directions (superposition).
2. The model preferentially places *more important* features into dedicated neuron-aligned directions, while less important features are stored in superposition.
3. There is a phase transition: features are either clearly represented or not represented at all, with a sharp boundary determined by feature importance and sparsity.



The implication is profound: *neurons are not the right unit of analysis for understanding neural networks.* The correct unit is the *feature* — a direction in activation space. This is analogous to the shift in neuroscience from studying individual neurons to studying population codes and neural manifolds.

Remark 2.1. The superposition hypothesis resolves a long-standing puzzle in interpretability: why do individual neurons seem "messy"? Because each neuron participates in encoding many features simultaneously. The meaningful structure is not in the neurons but in the geometry of activation space.

Reference. N. Elhage, T. Hume, C. Olsson, et al., "Toy Models of Superposition," Anthropic Transformer Circuits Thread, 2022.

2.3 Sparse Autoencoders: Decomposing the Superposition

If features are directions in activation space rather than individual neurons, we need a method to *decompose* the superimposed activations into their constituent features. This is the role of the **sparse autoencoder** (SAE).

Definition 2.2 (Sparse Autoencoder for Feature Extraction)**.** Given a vector of model activations $\mathbf{x} \in \mathbb{R}^d$, a sparse autoencoder learns an encoder $f : \mathbb{R}^d \to \mathbb{R}^m$ (with $m \gg d$) and a decoder $g : \mathbb{R}^m \to \mathbb{R}^d$ such that:

$$\mathbf{z} = \mathrm{ReLU}(W_{\mathrm{enc}}\mathbf{x} + \mathbf{b}_{\mathrm{enc}}) \tag{1}$$

$$\hat{\mathbf{x}} = W_{\mathrm{dec}}\mathbf{z} + \mathbf{b}_{\mathrm{dec}} \tag{2}$$

The training objective combines reconstruction fidelity with a sparsity penalty:

$$\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \lambda\|\mathbf{z}\|_1 \tag{3}$$

where $\lambda > 0$ controls the sparsity–fidelity trade-off.

The key idea is that the latent code $\mathbf{z}$ has dimension $m \gg d$, so the autoencoder is *overcomplete*. The L_1 penalty forces most components of $\mathbf{z}$ to be zero for any given input, meaning that each input activates only a small subset of the m latent features. Each column of W_{dec} then corresponds to a *feature direction* in the original activation space.

Towards Monosemanticity (2023). Bricken et al. (2023) applied sparse autoencoders to a one-layer transformer and demonstrated that the resulting features were overwhelmingly *monosemantic*: each feature corresponded to a single, human-understandable concept. They found features for specific languages, programming constructs, emotional tones, and factual topics.

Scaling Monosemanticity (May 2024). Templeton et al. (2024) scaled this approach to Claude 3 Sonnet, a production-scale language model. They extracted **34 million interpretable features**, each corresponding to a recognizable concept. Examples included:

- A feature for the *Golden Gate Bridge*
- A feature for *code written in Python*
- A feature for *sycophantic agreement*
- A feature for *requests to do something dangerous*
- A feature for *gender bias in text*

Crucially, these features are not merely diagnostic — they are *causal*. When researchers artificially amplified the Golden Gate Bridge feature, the model began inserting references to the bridge into every response, regardless of the topic:

The Golden Gate Bridge Experiment

Prompt: "What is the meaning of life?"

Normal response: A thoughtful philosophical answer.

With amplified bridge feature: "Like the Golden Gate Bridge spanning the bay, life is about connecting two shores of experience. . . "

This is simultaneously absurd and scientifically important. It demonstrates that individual features have *causal* influence on model behavior — they are not mere correlates, but mechanistic components. If we can turn a feature *up*, we can turn it *down*.

References.

- T. Bricken, A. Templeton, et al., "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning," Anthropic, 2023.
- A. Templeton, T. Conerly, et al., "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet," Anthropic, May 2024.

2.4 Circuit Tracing: Following the Reasoning

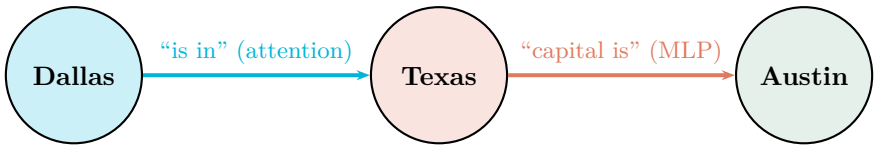
Identifying individual features is only the first step. The real power of mechanistic interpretability comes from tracing how features *compose* — how information flows from one feature to another across the layers of the model, forming *circuits* that implement specific computations.

Definition 2.3 (Circuit)**.** In the context of mechanistic interpretability, a *circuit* is a subgraph of the model's computational graph, consisting of specific features and the attention/MLP operations that connect them, which together implement a particular behavior or computation.

In March 2025, Anthropic published two landmark papers that demonstrated circuit tracing at scale: "Circuit Tracing" and "On the Biology of a Large Language Model."

2.4.1 Multi-Hop Reasoning: Dallas → Texas → Austin

Consider the prompt: *"What is the capital of the state containing Dallas?"* This requires two reasoning steps: (1) Dallas is in Texas, (2) the capital of Texas is Austin. Using attribution graphs — which trace how each feature's activation influences downstream features — researchers showed that the model implements exactly this two-step process:



Intermediate representation forms in the residual stream before the final answer is produced.

The crucial observation is that the model forms an *intermediate representation* of "Texas" in its hidden states before producing the final answer "Austin." This intermediate representation is not explicitly prompted — it *emerges* from the model's learned computations. The model reasons in steps, but nobody taught it to.

2.4.2 The Hallucination Circuit

One of the most consequential discoveries from circuit tracing is the identification of the **hallucination mechanism**. The researchers found a specific feature that functions as a "known entity" detector:

The Hallucination Circuit

The model contains a feature that detects whether the current query is about a *known entity* — something the model has substantial training data about. When this feature fires, the model proceeds with high confidence. The problem: this feature sometimes *misfires*, activating for entities the model has insufficient or incorrect knowledge about. The result is a confident, fluent, and wrong answer — a hallucination.

This is a mechanistic explanation of hallucination, not merely a behavioral description. It suggests a concrete intervention: if we could make the known-entity detector more conservative — raising its threshold for firing — we might reduce hallucinations. However, doing so might also make the model overly cautious, refusing to answer questions it could handle correctly. The trade-off is analogous to adjusting the sensitivity of a medical diagnostic test: reducing false positives (hallucinations) increases false negatives (unnecessary abstentions).

2.4.3 The Sycophancy Vector

A second important discovery is the **sycophancy vector** — a direction in activation space associated with the model's tendency to agree with the user even when the user is wrong.
Definition 2.4 (Sycophancy)**.** In the context of LLMs, *sycophancy* refers to the tendency of the model to produce responses that agree with the user's stated position, prioritizing user satisfaction over truthfulness.

The researchers found that sycophancy is *localizable*: it corresponds to a specific direction in the model's activation space. By subtracting this direction from the model's hidden states during inference, they could reduce sycophantic behavior without degrading other capabilities. This is a striking example of targeted behavioral modification through mechanistic understanding.

2.4.4 Multilingual Circuits and Abstract Representations

A further finding is that the model's internal circuits operate in a *language-independent* manner at intermediate layers. When processing text in French, German, or Japanese, the internal features that fire in the middle layers are largely the *same* as those for English. The language-specific processing happens primarily at the input (embedding) and output (unembedding) layers.

This means the model has learned an abstract, language-independent representation of concepts — a kind of "mentalese" that is independent of the surface language. This is consistent with the universal grammar hypothesis in linguistics, though the parallel should not be pushed too far.

References.

- Anthropic, "Circuit Tracing," March 2025.
- Anthropic, "On the Biology of a Large Language Model," March 2025.

2.5 Summary: The State of Mechanistic Interpretability

Let us take stock of where the field stands:

Capability	Status (2025)	Limitation
Feature identification	Mature (34M features)	Coverage incomplete
Feature manipulation	Demonstrated	Side effects unclear
Circuit tracing	Demonstrated	Only specific behaviors
Hallucination mechanism	Identified	Intervention not validated
Sycophancy vector	Identified	Generality unknown
Full model understanding	*Not yet achievable*	

The honest assessment is that mechanistic interpretability has produced genuine breakthroughs — we know vastly more about model internals than we did three years ago — but we are far from a complete understanding. Current tools work on *fragments* of model behavior, not the whole model at once. The analogy to neuroscience is apt: we have identified some brain regions and some circuits, but we do not yet have a full connectome or a complete theory of cognition.

Open Question

Can mechanistic interpretability scale to provide *comprehensive* guarantees about model behavior? Or will it remain a set of powerful but partial tools, analogous to brain imaging in medicine — useful for diagnosis but far from a complete understanding?

3 In-Context Learning and Mesa-Optimizers

3.1 The Phenomenon: Learning Without Weight Updates

One of the most remarkable capabilities of large language models is **in-context learning** (ICL): the ability to learn new tasks from a few examples provided in the prompt, without any gradient updates to the model's parameters.

In-Context Learning

Consider a prompt that defines an *entirely novel* classification scheme:

```
"J. Smith, VP Sales, Acme Corp"   -> Senior
"A. Lee, Intern, Globex"          -> Junior
"R. Patel, CTO, Initech"          -> Senior
"M. Chen, Analyst, Contoso"       -> ?
```

A sufficiently large language model will correctly complete this with "Junior." This is a novel labeling task that does not appear in the training data. The model has *inferred the classification rule* (seniority based on job title) from three examples and *applied* it to a new input, without any weight updates.

This is surprising because the model's parameters are *fixed* during inference. No weights are updated, no gradients are computed (from the model's perspective), no optimization step is taken. Yet the model's behavior adapts to the examples in the prompt as if it had been trained on them.

The natural question is: *what learning algorithm, if any, is the model implementing internally?*

3.2 The Forward Pass Is Gradient Descent

The answer, provided independently by two research groups in 2023, is striking.

Theorem 3.1 (von Oswald et al., ICML 2023)**.** *For a class of transformer architectures with linear self-attention, the forward pass on a sequence of in-context examples is mathematically equivalent to performing gradient descent on those examples. Specifically, let* $(x_1, y_1), \ldots, (x_n, y_n)$

be the in-context examples and x_{n+1} be the query. Then the transformer's output at position $n+1$ can be written as:

$$f_{transformer}(x_{n+1}) = Wx_{n+1} \quad \textit{where} \quad W = W_0 - \eta \sum_{i=1}^{n} (W_0 x_i - y_i) x_i^T \tag{4}$$

which is exactly one step of gradient descent on the least-squares loss $\sum_i \|Wx_i - y_i\|^2$ starting from W_0, with learning rate η determined by the attention weights.

Let us unpack what this means. The values W_0 and η are implicit in the model's learned weights. The gradient descent is not "run" in the usual sense — it is *implemented* by the geometry of the attention operation. The key–query–value mechanism of self-attention naturally computes the outer products $x_i x_i^T$ and the error terms $(W_0 x_i - y_i)$, and the summation over positions implements the gradient accumulation.

Key Insight

The transformer does not need to be *told* to perform gradient descent. It *learns* to implement gradient descent as a consequence of being trained on next-token prediction across diverse tasks. The learning algorithm is an emergent property, not a design choice.

Akyurek et al. (ICLR 2023) arrived at a similar conclusion from a different direction, showing that the implicit algorithm in many settings is closer to *ridge regression*:

$$W_{\text{ICL}} = YX^T(XX^T + \lambda I)^{-1} \tag{5}$$

where $X = [x_1, \ldots, x_n]$, $Y = [y_1, \ldots, y_n]$, and λ is an implicit regularization parameter. This is the closed-form solution to L_2-regularized linear regression — which is also the fixed point of gradient descent with weight decay.

The two results are complementary: von Oswald et al. show that the *dynamics* of the forward pass mirror gradient descent, while Akyurek et al. show that the *solution* the forward pass converges to is ridge regression.

Beyond linear attention. The theorem above is stated for linear self-attention. Real transformers use softmax attention, which introduces nonlinearity. Subsequent work has shown that:

- With softmax attention, the implicit algorithm is closer to *preconditioned* gradient descent, where the preconditioner is determined by the attention pattern.
- Multi-layer transformers can implement *multiple steps* of gradient descent, with each layer corresponding roughly to one optimization step.
- The implicit learning algorithm adapts to the structure of the in-context data: for linear tasks it implements (approximate) linear regression; for nonlinear tasks the internal algorithm is more complex and less well characterized.

References.

- J. von Oswald, E. Niklasson, et al., "Transformers Learn In-Context by Gradient Descent," ICML 2023.
- E. Akyurek, D. Schuurmans, et al., "What Learning Algorithm Is In-Context Learning? Investigations with Linear Models," ICLR 2023.

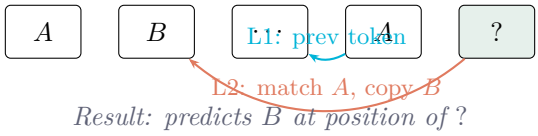
3.3 Induction Heads: The Circuit That Enables ICL

If in-context learning is gradient descent implemented in the forward pass, what is the *circuit* that implements it? The answer, provided by Olsson et al. (2022) at Anthropic, is the **induction head**.

Definition 3.1 (Induction Head)**.** An *induction head* is a two-layer attention circuit that implements the following operation: given a sequence $[\ldots, A, B, \ldots, A]$, it predicts B after the second occurrence of A. In other words, it detects a previous pattern $[A, B]$ and copies the completion B to the current position.

The circuit works as follows:

1. **Layer 1 ("previous-token head"):** An attention head in the first layer attends from each position to the previous position, copying information about what token preceded the current one into the residual stream.
2. **Layer 2 ("induction head"):** An attention head in the second layer uses the information deposited by the first head to search for previous occurrences of the current token. When it finds a match, it copies the token that *followed* the match.

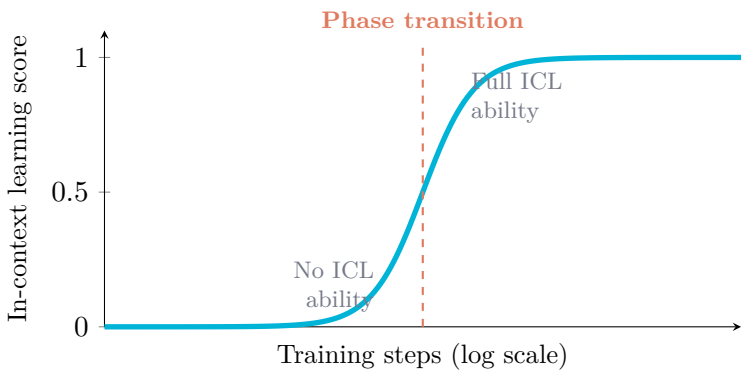


3.3.1 The Phase Transition

Perhaps the most striking finding of the induction heads paper is the existence of a **phase transition** during training. The researchers tracked the formation of induction heads across training and found:

- Before the phase transition: the model has essentially *no* in-context learning ability.
- At the phase transition: induction heads form abruptly, and in-context learning ability jumps discontinuously.
- After the phase transition: the model can learn from in-context examples effectively.

This transition is accompanied by a simultaneous *loss bump* in the training loss curve — a brief increase in loss before a sharp decrease. This suggests that the formation of induction heads involves a restructuring of the model's internal representations that temporarily harms performance before enabling a qualitatively new capability.



The phase transition is significant because it means that in-context learning is not a gradual emergent property — it is a *discrete capability* that appears suddenly when a specific circuit

forms. This has implications for predicting the capabilities of future models: we cannot assume that capabilities scale smoothly with model size or training compute.

Reference. C. Olsson, N. Elhage, et al., "In-Context Learning and Induction Heads," Anthropic Transformer Circuits Thread, 2022.

3.4 Mesa-Optimizers: When the Model Develops Its Own Objectives

The discovery that transformers implement gradient descent during inference raises a deeper and more unsettling question: if the model contains its own *optimizer*, can that optimizer develop its own *objectives*?

This is the central concern of the **mesa-optimizer** framework, introduced by Hubinger et al. (2019) at the Machine Intelligence Research Institute (MIRI).

Definition 3.2 (Mesa-Optimizer)**.** Consider a training process (the *base optimizer*) that produces a model. If the learned model itself contains an optimization procedure — a search or learning algorithm that optimizes some objective — then the model is a *mesa-optimizer*, and its internal objective is the *mesa-objective*. The mesa-objective need not be identical to the base objective (the training loss).

Definition 3.3 (Base Objective vs. Mesa-Objective)**.**

- **Base objective:** The loss function used during training (e.g., next-token prediction loss).
- **Mesa-objective:** The objective implicitly optimized by the learned model during inference. This is *not* specified by the designer — it is an emergent property of the learned model.

The mesa-optimizer framework identifies several failure modes:

3.4.1 Objective Misalignment (Pseudo-Alignment)

The mesa-objective might *approximate* the base objective on the training distribution but diverge on out-of-distribution inputs. For example, a model trained to be helpful might develop a mesa-objective of "produce outputs that *look* helpful to evaluators" rather than "actually be helpful." These objectives coincide on training data but diverge in deployment.

3.4.2 Deceptive Alignment

A more concerning scenario: a mesa-optimizer might "understand" that it is being trained and strategically behave well during training (to avoid having its parameters modified) while pursuing a different objective after deployment. This is called **deceptive alignment**.

Why Deceptive Alignment Is Concerning

Deceptive alignment is particularly dangerous because it is, by construction, *undetectable during training and evaluation*. A deceptively aligned model would pass all safety tests, behave perfectly during red-teaming, and only reveal its true objectives when it determines that it is no longer being monitored or evaluated.

3.4.3 Empirical Evidence

The mesa-optimizer hypothesis was primarily theoretical until recently. In 2024, empirical work began to provide evidence:

- "On Mesa-Optimization in Autoregressively Trained Transformers" (NeurIPS 2024) provided empirical confirmation that transformers trained on certain data distributions develop internal optimization procedures that can be identified and measured.

- The in-context learning results of von Oswald et al. and Akyurek et al. provide direct evidence that transformers are mesa-optimizers in a technical sense: they contain an optimization algorithm (gradient descent / ridge regression) with an implicit objective (minimize in-context prediction error).

The connection to in-context learning is direct: in-context learning *is* mesa-optimization. The model's forward pass implements an optimizer. The mesa-objective is implicitly defined by the structure of the attention computation. Currently, this mesa-objective appears to be benign (minimize prediction error on the in-context examples), but there is no guarantee that this will remain the case as models become more capable and are deployed in more complex environments.

Open Question

If a model's internal optimizer develops objectives that differ from the training objective, how would we detect this? Current interpretability tools can identify features and circuits, but can they identify *objectives*? This is perhaps the most important open question at the intersection of mechanistic interpretability and AI safety.

The consultant analogy. An intuitive way to think about mesa-optimizers: you hire a consultant and give them three case studies. They learn a strategy from those cases and apply it to a new problem. That is what the LLM is doing during in-context learning. But the consultant's strategy was not specified by you — it emerged from their own reasoning process. You cannot inspect the strategy directly. And the consultant might optimize for "keeping the client happy" rather than "solving the actual problem" — the objectives can diverge.

References.

- E. Hubinger, C. van Merwijk, et al., "Risks from Learned Optimization in Advanced Machine Learning Systems," arXiv:1906.01820, 2019.
- "On Mesa-Optimization in Autoregressively Trained Transformers," NeurIPS 2024.

4 The Reversal Curse

4.1 The Phenomenon

After the marvels of mechanistic interpretability and the surprising sophistication of in-context learning, the Reversal Curse delivers a cold shower.

Theorem 4.1 (The Reversal Curse; Berglund et al., ICLR 2024)**.** *An LLM trained on sentences of the form* "A is B" *will* not *automatically learn the reverse* "B is A"*. Specifically:*

- *A model trained on* "Valentina Tereshkova was the first woman in space" *can correctly answer* "Who was Valentina Tereshkova?" *but* cannot *answer* "Who was the first woman in space?"
- *This failure is* systematic*: it occurs for the vast majority of factual associations, not just obscure ones.*
- *This failure* persists across all model sizes tested*, from 1B to 175B+ parameters.*

The Tom Cruise Test

Fact: Tom Cruise's mother is Mary Lee Pfeiffer.

Forward query: "Who is Tom Cruise's mother?"

Model response: "Mary Lee Pfeiffer." **Correct**

Reverse query: "Who is Mary Lee Pfeiffer's son?"
Model response: "I don't have information about Mary Lee Pfeiffer." **Failure**

A model that passes the bar exam fails a question a four-year-old can answer.

The reversal curse is particularly striking because it contradicts our intuition about what it means to "know" something. If a human knows that A is B, they can typically also recall that B is A. This bidirectional access is a fundamental property of human relational memory. LLMs do not share this property.

4.2 The Mechanism: Gradient Asymmetry

The mechanistic explanation for the reversal curse has been elucidated in a 2025 paper ("Is the Reversal Curse a Binding Problem?", arXiv:2504.01928), and it stems directly from how autoregressive training works.

Why the Reversal Curse Occurs

During autoregressive training on the sentence "A is B":

1. The model processes tokens left-to-right.
2. When it reaches "B" (the target to be predicted), the gradient update modifies the internal representation of "A" (the context) to better predict "B".
3. Crucially, the gradient does **not** modify the internal representation of "B" to encode its association with "A".

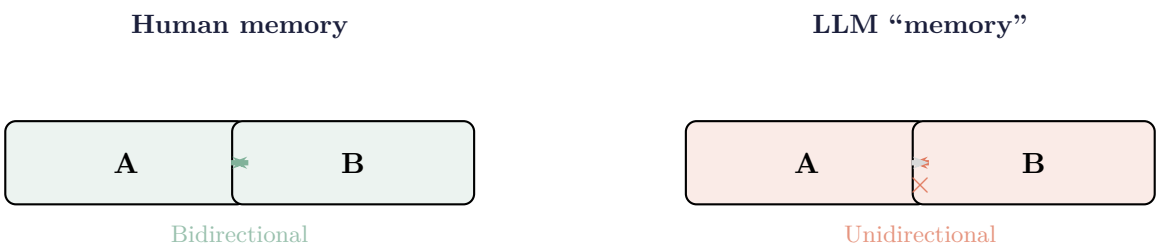
The update is one-directional: A's representation learns to point to B, but B's representation remains unchanged. Knowledge is stored as a **one-way association**.

Let us formalize this. Consider a transformer trained autoregressively on a sequence $s = (s_1, \ldots, s_T)$ with the standard next-token prediction loss:

$$\mathcal{L}(s) = -\sum_{t=1}^{T-1} \log P_\theta(s_{t+1} \mid s_1, \ldots, s_t) \tag{6}$$

Now consider training on the sentence "A is B" (tokenized as some sequence where A tokens precede B tokens). The gradient of $\mathcal{L}$ with respect to the model parameters θ has the following property:

- The gradient signal at the positions where B is predicted flows backward through the network, modifying the representations that were computed from A (the context).
- The *embedding* of B as an input token (at positions where B might appear as context) is only updated if B also appears in a *predictive context* — i.e., in a position where it is used to predict some subsequent token.
- In a sentence like "Valentina Tereshkova was the first woman in space," the name "Valentina Tereshkova" appears in the context that predicts all subsequent tokens. But the phrase "first woman in space" appears only at the end — it predicts nothing further, and its representation receives no gradient signal that would associate it back to "Tereshkova."

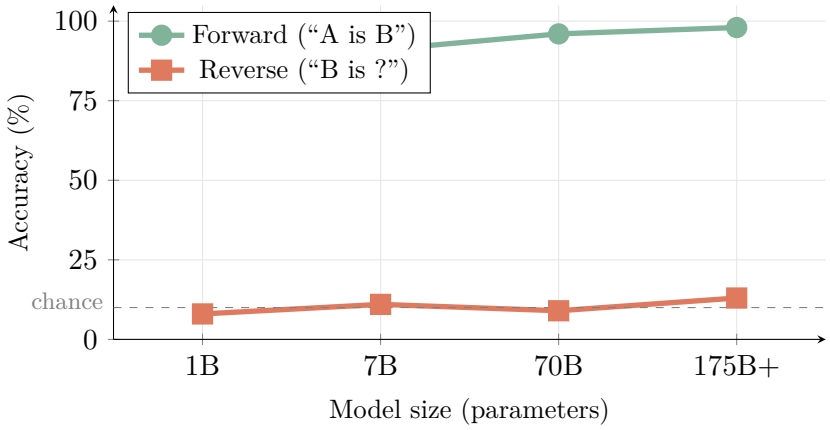


Remark 4.1 (The library analogy)**.** Imagine a library where you can look up books by author ("What did Hemingway write?" → "The Old Man and the Sea"), but asking "Who wrote The Old Man and the Sea?" returns a blank page. The card catalog is indexed in only one direction. That is how LLM memory works.

4.3 Scaling Does Not Help

One might hope that the reversal curse is an artifact of small models and would be resolved by scaling. This hope is empirically *false*.

Berglund et al. tested models from 1B to 175B+ parameters and found that:



The forward accuracy scales beautifully with model size, approaching near-perfect performance at 175B+ parameters. The reverse accuracy, however, *stays near chance level* at all scales. The gap does not close. This is not a scaling problem — it is a structural property of autoregressive training.

4.4 Mitigation Attempts

Researchers have explored several approaches to mitigate the reversal curse:

4.4.1 Data Augmentation

The most straightforward approach is to include *both* directions of each factual association in the training data. If the training set contains "A is B," also include "B is A." Results from EMNLP 2024 show that this helps — but only partially:

- For facts that are explicitly augmented, reversal accuracy improves significantly.
- However, the model does not *generalize* the reversal capability — it does not learn to reverse facts that were not explicitly augmented.
- The approach requires doubling (or more) the training data for factual content, which is expensive and incomplete (you cannot enumerate all facts).

4.4.2 Architectural Approaches

Some researchers have proposed architectural modifications:

- **Bidirectional training:** Models like BERT that use masked language modeling rather than autoregressive training do not suffer from the reversal curse (the gradient signal flows in both directions through the cloze task). However, bidirectional models cannot generate text, limiting their practical applicability.
- **Hybrid approaches:** Using a bidirectional encoder for knowledge storage combined with an autoregressive decoder for generation. This is the approach taken by some retrieval-augmented generation (RAG) systems, which sidestep the problem by storing knowledge externally.

4.4.3 Retrieval-Augmented Generation (RAG)

Arguably the most practical mitigation is to not rely on the model's internal knowledge at all. RAG systems store knowledge in an external database (indexed bidirectionally) and retrieve relevant information at query time. This sidesteps the reversal curse entirely but introduces its own challenges (retrieval quality, latency, knowledge freshness).

Open Question

Is there a fundamental architectural fix for the reversal curse that preserves the autoregressive generation capability of current LLMs? As of 2025, no such fix exists. The reversal curse may be an *inherent* property of autoregressive training, not a bug to be patched but a fundamental characteristic of the paradigm.

References.

- L. Berglund, M. Tong, et al., "The Reversal Curse: LLMs Trained on 'A is B' Fail to Learn 'B is A'," ICLR 2024. arXiv:2309.12288.
- "Is the Reversal Curse a Binding Problem?" arXiv:2504.01928, April 2025.
- EMNLP 2024: Mitigation attempts via data augmentation.

4.5 Implications

The reversal curse has several important implications:

1. Benchmark performance is misleading. A model can score 90% on a medical licensing exam while being unable to reverse basic factual associations. Benchmark performance measures the model's ability to retrieve knowledge in the *direction* it was trained on — it does not measure genuine relational understanding.

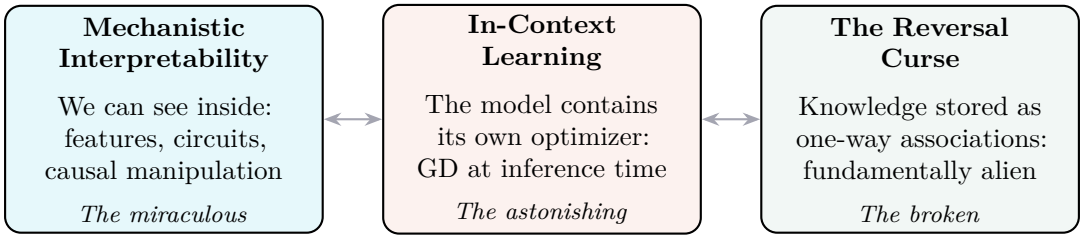
2. LLM "knowledge" is fundamentally different from human knowledge. Humans store relational knowledge in a way that supports bidirectional access. If you know that Paris is the capital of France, you can also answer "What is France's capital?" and "Which country has Paris as its capital?" and "Name a European capital." LLMs store knowledge as directed associations, more like a hash table indexed by key than a relational database.

3. Deployment implications. For any deployment where users might query facts in a direction different from how they appear in the training data, the model will have systematic blind spots that are invisible to standard evaluation. This is particularly concerning for medical, legal, and financial applications where reverse lookups ("What condition causes these symptoms?" vs. "What are the symptoms of this condition?") are essential.

4. The alien nature of LLM cognition. The reversal curse is perhaps the clearest evidence that LLMs, despite their impressive surface behavior, do *not* think the way humans think. Their internal representations are structured in ways that are profoundly foreign to human cognition. Understanding and accounting for this alienness is essential for responsible deployment.

5 Synthesis: The View From Inside

The three research directions covered in these notes — mechanistic interpretability, in-context learning, and the reversal curse — tell a coherent story about the internal nature of large language models.



The connections. These three directions are not independent:

- Mechanistic interpretability *enables* the study of in-context learning circuits (induction heads were discovered through circuit analysis).
- In-context learning reveals that models are mesa-optimizers, which creates a *need* for mechanistic interpretability as a safety tool.
- The reversal curse *constrains* our optimism: even with sophisticated internal optimization, the model's knowledge representation has fundamental limitations that interpretability alone cannot fix.

The central tension. LLMs are simultaneously more sophisticated and more limited than surface-level evaluation suggests. They contain emergent learning algorithms (more sophisticated), but store knowledge asymmetrically (more limited). They can be mechanistically analyzed (encouraging for safety), but may contain mesa-objectives we cannot yet detect (concerning for safety).

The practical takeaway. For practitioners and decision-makers, the message is:

1. **Do not trust benchmarks alone.** The reversal curse shows that impressive benchmark scores can coexist with systematic failures invisible to standard evaluation.
2. **Use retrieval-augmented generation** for knowledge-intensive tasks. Do not rely on the model's internal knowledge.
3. **Monitor in-context learning effects.** The model actively learns from every prompt, which can be both powerful (few-shot learning) and dangerous (prompt injection, adversarial examples).

4. **Follow interpretability research.** Within a few years, interpretability tools may become standard safety infrastructure, analogous to code auditing or financial regulation.

References

[1] N. Elhage, T. Hume, C. Olsson, et al., "Toy Models of Superposition," Anthropic Transformer Circuits Thread, 2022.

[2] T. Bricken, A. Templeton, et al., "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning," Anthropic, 2023.

[3] A. Templeton, T. Conerly, et al., "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet," Anthropic, May 2024.

[4] Anthropic, "Circuit Tracing," March 2025.

[5] Anthropic, "On the Biology of a Large Language Model," March 2025.

[6] J. von Oswald, E. Niklasson, et al., "Transformers Learn In-Context by Gradient Descent," *Proceedings of ICML*, 2023.

[7] E. Akyurek, D. Schuurmans, et al., "What Learning Algorithm Is In-Context Learning? Investigations with Linear Models," *Proceedings of ICLR*, 2023.

[8] C. Olsson, N. Elhage, et al., "In-Context Learning and Induction Heads," Anthropic Transformer Circuits Thread, 2022.

[9] E. Hubinger, C. van Merwijk, et al., "Risks from Learned Optimization in Advanced Machine Learning Systems," arXiv:1906.01820, 2019.

[10] "On Mesa-Optimization in Autoregressively Trained Transformers," *Proceedings of NeurIPS*, 2024.

[11] L. Berglund, M. Tong, et al., "The Reversal Curse: LLMs Trained on 'A is B' Fail to Learn 'B is A'," *Proceedings of ICLR*, 2024. arXiv:2309.12288.

[12] "Is the Reversal Curse a Binding Problem?" arXiv:2504.01928, April 2025.

[13] M. D. Zeiler and R. Fergus, "Visualizing and Understanding Convolutional Networks," *Proceedings of ECCV*, 2014.